TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

KỸ THUẬT

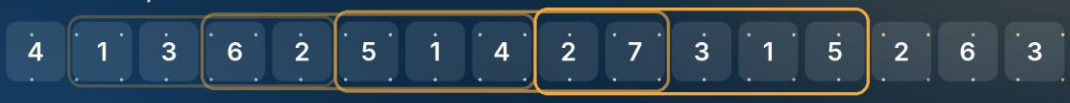
HAI CON TRỎ

& cửa sổ trượt

Từ cơ bản đến nâng cao



CỬA SỔ TRƯỢT



TẬP 1 LÝ THUYẾT

14 bài học · 4 chương · Hơn 80 hình minh họa

NỘI DUNG

1. NỀN TẢNG HAI CON TRỎ
2. CỬA SỔ TRƯỢT
3. HAI CON TRỎ KẾT HỢP
4. HAI CON TRỎ NÂNG CAO

Tổng hợp và biên soạn: **Lê Thành Nam**

Lê Thành Nam

LỜI NÓI ĐẦU

Trong các đề thi học sinh giỏi Tin học, có một kĩ thuật xuất hiện với tần suất rất cao nhưng lại hiếm khi được dạy một cách bài bản: **kĩ thuật hai con trỏ**. Nó không có công thức phức tạp, không cần cấu trúc dữ liệu cầu kì, chương trình thường chỉ dài mươi dòng. Thế nhưng chính mươi dòng ấy biến một lời giải $O(n^2)$ chỉ ăn được subtask nhỏ thành một lời giải $O(n)$ ăn trọn điểm. Nhiều học sinh đọc lời giải hai con trỏ thì thấy "dễ quá", nhưng tự gặp bài mới lại không nhận ra, hoặc nhận ra nhưng viết sai ở một điều kiện dừng, một phép cộng tràn số, một lần di chuyển con trỏ sai thứ tự.

Nguyên nhân là hai con trỏ không phải một thuật toán để học thuộc, mà là một **cách nhìn**: Nhận ra rằng khi một đầu của đoạn đang xét dịch chuyển, đầu kia chỉ cần dịch chuyển theo một chiều; nhận ra rằng có những phương án có thể bị loại bỏ cả loạt mà không cần thử; và chứng minh được rằng việc loại bỏ ấy không làm mất đáp án. Bộ tài liệu này được viết để giúp em hình thành cách nhìn ấy một cách có hệ thống, từ bài toán tìm hai số có tổng cho trước đến các kĩ thuật kết hợp ở trình độ tiệm cận học sinh giỏi quốc gia.

Bộ sách gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** (cuốn sách này) gồm 14 bài học chia thành 4 chương: Nền tảng hai con trỏ; Cửa sổ trượt; Hai con trỏ kết hợp với sắp xếp, tìm kiếm nhị phân, vòng tròn và chia đôi tập hợp; Hai con trỏ nâng cao với hàng đợi đơn điệu, hàng đợi hai ngăn xếp và bảng hai chiều.
- **Tập 2: Bài tập** gồm 42 bài, mỗi bài học có ba bài tập ở ba mức độ (cơ bản, trung bình, nâng cao), có ràng buộc chia theo subtask như đề thi thật.
- **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm sơ đồ khối, chương trình hoàn chỉnh và bảng điểm thực tế của từng cách làm.

Mỗi bài học không đưa ra thuật toán ngay. Em sẽ được đặt trước một bài toán cụ thể, thử cách ngây thơ, đếm số phép tính để thấy chỗ bế tắc, rồi được dẫn từng bước để phá bế tắc. Mọi thuật toán đều được **chạy tay** trên hình vẽ từng lượt, có lập luận **vì sao đúng**, và có phần **đếm số phép tính** giải thích rõ vì sao độ phức tạp là $O(n)$ chứ không phải $O(n^2)$, điều mà nhiều tài liệu bỏ qua nhưng lại là chỗ học sinh hay hiểu sai nhất về hai con trỏ.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Mọi chương trình trong sách đều đã được kiểm tra bằng cách đối chiếu với lời giải vét cạn trên hàng nghìn bộ dữ liệu ngẫu nhiên.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi thật sự suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, gần gũi để em bắt đầu nghĩ.
Bài toán	Đề bài cụ thể, có ví dụ và hình minh họa.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm sơ đồ khối và phần đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm, và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: quan sát, nhận xét, thuật toán, chạy tay, chứng minh, đếm phép tính, chương trình.
Mở rộng	Các biến thể thường gặp trong đề thi, mỗi biến thể có ví dụ và mã nguồn.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
NHẬN XÉT / Ý TƯỞNG	Quan sát then chốt dẫn tới thuật toán.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp, giải thích vì sao là $O(n)$, $O(n^2)$, $O(n \log n)$, ...
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn.
KHUÔN MẪU	Đoạn mã khung có thể dùng lại cho cả một lớp bài.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Quy ước màu trong các hình vẽ chạy tay cũng được giữ nguyên suốt cuốn sách:

Lê Thành Nam

Màu ô	Ý nghĩa
Cam	Vị trí con trỏ trái (L, i, l).
Xanh dương	Vị trí con trỏ phải (R, j, r).
Xanh lá	Phần tử nằm trong cửa sổ đang xét, hoặc cặp thỏa mãn.
Xám	Phần tử đã bị loại, sẽ không bao giờ được xét lại.
Vàng	Phần tử đang được chú ý đặc biệt ở lượt đó.
Đỏ nhạt	Phần tử hoặc cặp vi phạm điều kiện.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp một hình chạy tay, hãy che phần ghi chú bên phải lại và tự quyết định con trỏ nào di chuyển, rồi mới mở ra đối chiếu. Mỗi khi gặp một thuật toán hai con trỏ, hãy tự trả lời ba câu hỏi: **Mỗi con trỏ đi theo chiều nào và vì sao không bao giờ phải quay lại? Điều gì luôn đúng sau mỗi lượt (bất biến)? Vì sao tổng số lần di chuyển chỉ là O(n)?** Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

CHƯƠNG 1. NỀN TẢNG HAI CON TRỎ 6
Bài 1. Hai con trỏ đi ngược chiều 6
Bài 2. Hai con trỏ cùng chiều trên hai dãy 6
Bài 3. Con trỏ đọc và con trỏ ghi 6

CHƯƠNG 2. CỬA SỔ TRƯỢT 60
Bài 4. Cửa sổ độ dài cố định 60
Bài 5. Cửa sổ co giãn và tính đơn điệu 60
Bài 6. Cửa sổ với mảng đếm 60
Bài 7. Đếm đoạn "đúng K": Hiệu của hai lần "không quá" 96

CHƯƠNG 3. HAI CON TRỎ KẾT HỢP 107
Bài 8. Sắp xếp rồi dùng hai con trỏ: Bộ ba và tam giác 108
Bài 9. Tìm kiếm nhị phân đáp án, đếm bằng hai con trỏ 107
Bài 10. Hai con trỏ trên vòng tròn 107
Bài 11. Chia đôi tập hợp và hai con trỏ 136

CHƯƠNG 4. HAI CON TRỎ NÂNG CAO 144
Bài 12. Cửa sổ có max và min: Hàng đợi đơn điệu 144
Bài 13. Phép toán không đảo được: Hàng đợi hai ngăn xếp 144
Bài 14. Hai con trỏ trên bảng hai chiều 144

TỔNG KẾT: BẢN ĐỒ KĨ THUẬT HAI CON TRỎ 172

Lê Thành Nam

CHƯƠNG 1. NỀN TẢNG HAI CON TRỎ

Khi lần đầu học lập trình, em quen với việc dùng hai vòng lặp lồng nhau để xét mọi cặp phần tử. Cách làm ấy đúng, dễ viết, nhưng với một triệu phần tử nó cần năm trăm tỉ phép thử. Chương này dạy em nhìn bài toán theo một cách khác: Đặt hai "ngón tay" lên dãy số và để chúng tự tìm đường. Mỗi lần một ngón tay dịch chuyển là một lần ta loại bỏ được cả một loạt phương án mà không cần thử. Nhờ vậy, hai vòng lặp lồng nhau với n^2 bước biến thành một vòng lặp chỉ với chưa đến 2n bước.

Ba bài học của chương tương ứng với ba kiểu chuyển động cơ bản của hai con trỏ: Đi ngược chiều từ hai đầu dãy; đi cùng chiều trên hai dãy khác nhau; và đi cùng chiều trên cùng một dãy, một con trỏ đọc và một con trỏ ghi. Mọi kĩ thuật ở các chương sau đều được ghép từ ba kiểu chuyển động này.

TRONG CHƯƠNG NÀY

Bài 1. Hai con trỏ đi ngược chiều: Tổng hai số, đếm cặp, bể chứa nước.

Bài 2. Hai con trỏ cùng chiều trên hai dãy: Trộn, giao, dãy con và nguyên lí "con trỏ không lùi".

Bài 3. Con trỏ đọc và con trỏ ghi: Lọc dãy tại chỗ, phân hoạch, cờ ba màu, rùa và thỏ.

Lê Thành Nam

BÀI 1 Hai con trỏ đi ngược chiều

Học xong bài này, em sẽ:

✓ Thấy tận mắt vì sao cách thử mọi cặp cần tới $n(n-1)/2$ phép thử, và vì sao đó là con số không thể chấp nhận với $n = 10^6$.

✓ Hiểu nguyên lí cốt lõi của hai con trỏ: Mỗi phép so sánh **loại bỏ cả một hàng hoặc một cột** phương án, không chỉ một phương án.

✓ Chứng minh được thuật toán hai con trỏ không bỏ sót đáp án bằng bảng cặp và lập luận "phần tử vô dụng".

✓ Giải được ba dạng: Tìm một cặp có tổng bằng S, đếm số cặp có tổng không vượt quá S, đếm số cặp có tổng đúng bằng S khi dãy có phần tử trùng nhau.

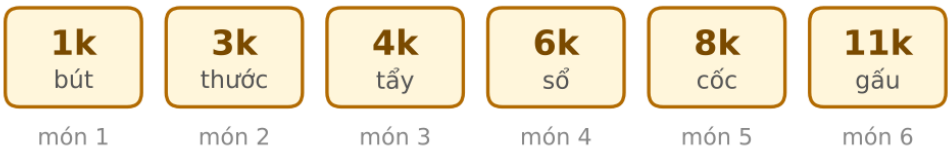
✓ Vận dụng được ý tưởng loại bỏ cho bài toán bể chứa nước, nơi không có tìm kiếm nhị phân nào giúp được.

Lê Thanh Nam

KHỞI ĐỘNG

Quầy lưu niệm của trường bày sáu món quà, giá đã được xếp tăng dần: 1, 3, 4, 6, 8, 11 (nghìn đồng). Em có đúng 10 nghìn đồng và muốn mua **đúng hai món** sao cho tiêu vừa hết tiền. Em sẽ chọn hai món nào?

Có lẽ em sẽ nhẩm thử vài cặp và nhanh chóng tìm ra 4 + 6 = 10. Nhưng hãy tưởng tượng quầy có một triệu món. Nếu thử lần lượt từng cặp, em cần thử khoảng năm trăm tỉ cặp. Cuối bài này, em sẽ tìm ra cặp cần mua chỉ với chưa đầy một triệu phép so sánh, và hiểu tại sao không cần thử những cặp còn lại.



Hình 1.1. *Sáu món quà với giá đã sắp xếp tăng dần (đơn vị: nghìn đồng)*

BÀI TOÁN. TỔNG HAI SỐ [TONGHAI]

Cho dãy n số nguyên đã được sắp xếp không giảm $a_1 \le a_2 \le ... \le a_n$ và một số nguyên S. Hãy tìm hai chỉ số $i < j$ sao cho $a_i + a_j = S$.

Dữ liệu vào: Dòng đầu gồm hai số n, S ($1 \le n \le 10^6$, $|S| \le 2 \cdot 10^9$). Dòng thứ hai gồm n số $a_1, a_2, ..., a_n$ ($|a_i| \le 10^9$), đã sắp xếp không giảm.

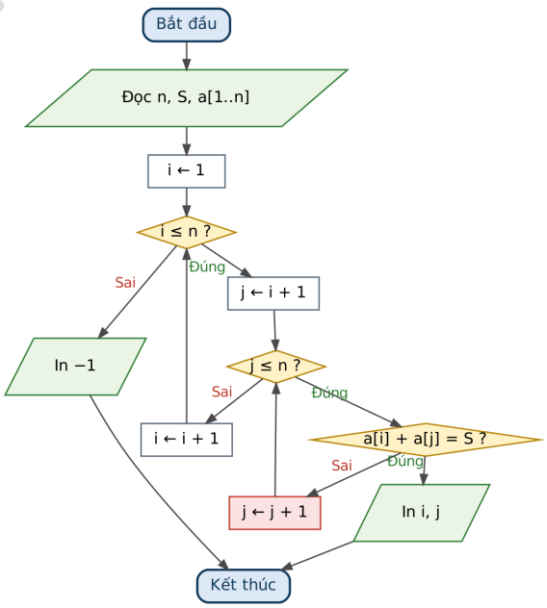
Kết quả: Hai chỉ số i, j tìm được (nếu có nhiều cặp, in một cặp bất kì); nếu không có cặp nào, in −1.

TONGHAI.INP	TONGHAI.OUT	Giải thích
6 10 1 3 4 6 8 11	3 4	4 + 6 = 10. Đây là cặp duy nhất.
6 12 1 3 4 6 8 11	1 6	1 + 11 = 12. Cặp (3, 5) với 4 + 8 = 12 cũng đúng.
6 20 1 3 4 6 8 11	-1	Tổng lớn nhất là 8 + 11 = 19 < 20.

Lưu ý dữ liệu: Mỗi số có trị tuyệt đối tới 10^9 nên tổng hai số có thể tới $2 \cdot 10^9$, sát với giới hạn $2^{31} - 1 \approx 2{,}147 \cdot 10^9$ của kiểu int. Chỉ cần đề nới ràng buộc một chút là tràn số. Thói quen an toàn: Dùng kiểu long long cho mọi giá trị và mọi phép cộng.

1. Ý tưởng ngây thơ: Thử mọi cặp

Cách tự nhiên nhất là xét tất cả các cặp chỉ số (i, j) với $i < j$: Cố định i, cho j chạy từ i + 1 tới n, gặp cặp nào có tổng bằng S thì in ra và dừng.



Hình 1.2. Sơ đồ khối của cách thử mọi cặp. Ô đỏ là bước được lặp nhiều nhất

Lê Thanh Nam

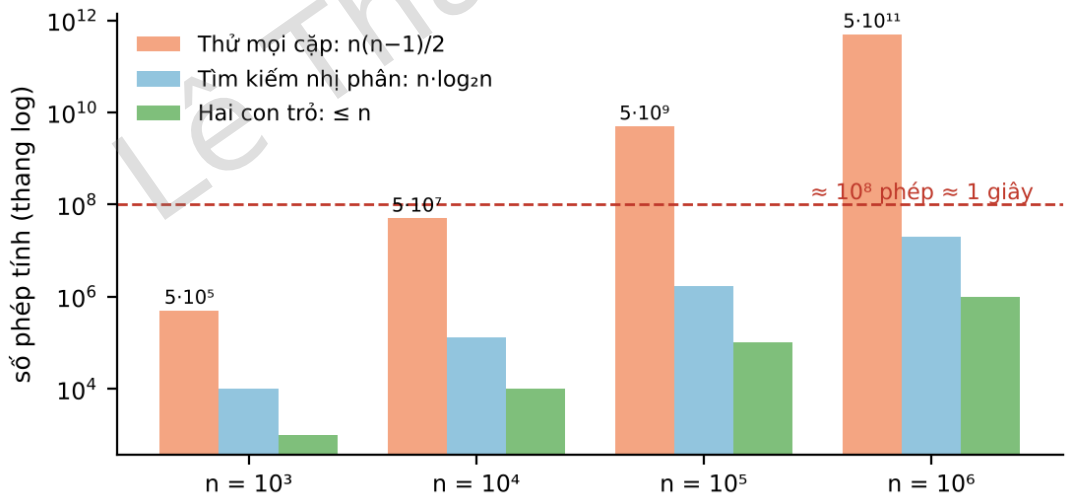
Code ý chính: Thử mọi cặp

```
for (int i = 1; i <= n; i++)
    for (int j = i + 1; j <= n; j++)
        if (a[i] + a[j] == s) {
            cout << i << ' ' << j << '\n';
            return 0;
        }
cout << -1 << '\n';
```

Chương trình đúng với cả ba ví dụ. Hãy để ý rằng cách làm này **không hề dùng tới** việc dãy đã được sắp xếp: Nó chạy y hệt trên một dãy lộn xộn. Một thông tin quý giá của đề bài đã bị bỏ phí. Trước khi đi tìm cách tận dụng thông tin ấy, ta đếm xem cách ngây thơ tốn bao nhiêu phép tính.

VÌ SAO CÁCH NGÂY THƠ CHẬM? ĐẾM TỪNG BƯỚC

- Với i = 1, vòng trong chạy n − 1 lần; với i = 2, chạy n − 2 lần; ...; với i = n − 1, chạy 1 lần. Trong trường hợp xấu nhất (không có cặp nào thỏa mãn), tổng số lần thử là (n − 1) + (n − 2) + ... + 1 = **n(n − 1)/2**.
- Con số này xấp xỉ $n^2/2$, nên độ phức tạp là **O(n^2)**: Khi n tăng gấp 10 lần, thời gian tăng gấp 100 lần.
- Đo thực tế: n = 10^3 cần $5 \cdot 10^5$ phép thử (tức thì); n = 10^4 cần $5 \cdot 10^7$ (khoảng nửa giây); n = 10^5 cần $5 \cdot 10^9$ (khoảng một phút); n = 10^6 cần $5 \cdot 10^{11}$ phép thử, tức **hơn một giờ**.



Hình 1.3. Số phép tính của ba cách làm theo n (thang logarit). Đường đỏ là ngưỡng khoảng 10^8 phép tính mỗi giây

Lê Thành Nam

BẾ TẮC

Với $n = 10^6$, phải thử $5 \cdot 10^{11}$ cặp. Thế nhưng dãy đã được sắp xếp: Biết $a_i + a_j$ quá lớn thì cặp $(i, j + 1)$ chắc chắn còn lớn hơn. Cách ngây thơ vẫn thử nó. Làm sao để **mỗi phép so sánh cho ta biết nhiều hơn** một cặp đúng hay sai?

2. Một bước cải tiến: Tìm kiếm nhị phân

Ý tưởng tận dụng thứ tự đầu tiên mà nhiều em nghĩ tới: Cố định i, ta cần tìm xem trong đoạn a_{i+1}, ..., a_n có số nào **bằng đúng** $S - a_i$ không. Đoạn này đã sắp xếp, nên ta tìm kiếm nhị phân, mỗi lần tốn khoảng $\log_2 n$ bước.

Code ý chính: Cố định i, tìm nhị phân giá trị S − a[i]

```
for (int i = 1; i < n; i++) {
    long long need = s - a[i];
    int j = lower_bound(a.begin() + i + 1, a.begin() + n + 1, need) - a.begin();
    if (j <= n && a[j] == need) {
        cout << i << ' ' << j << '\n';
        return 0;
    }
}
cout << -1 << '\n';
```

Mỗi lần tìm nhị phân tốn khoảng $\log_2(10^6) \approx 20$ bước, nên tổng cộng khoảng $n \cdot \log_2 n \approx 2 \cdot 10^7$ bước: Độ phức tạp **O(n log n)**, đủ nhanh cho $n = 10^6$. Bài toán xem như đã giải xong?

Chưa hẳn. Cách này còn ba điểm chưa thật đẹp. Thứ nhất, nó vẫn "phí": Với mỗi i ta tìm nhị phân lại từ đầu, không dùng gì từ những lần tìm trước, trong khi khi i tăng thì giá trị cần tìm $S - a_i$ giảm dần, tức là vị trí cần tìm chỉ lùi dần về bên trái. Thứ hai, nó chỉ dùng được khi ta tìm **đúng một giá trị**; ở các bài toán như bể chứa nước cuối bài, không có giá trị cụ thể nào để tìm. Thứ ba, khi n lên tới 10^7 (dữ liệu sinh bằng công thức), hệ số log bắt đầu đáng kể. Có cách nào chỉ cần **O(n)** và đơn giản hơn không?

CÂU HỎI DẪN ĐƯỜNG

Thay vì cố định một số rồi đi tìm bạn của nó, hãy nhìn vào **hai số ở hai đầu dãy**. Chúng là số nhỏ nhất và số lớn nhất. So sánh tổng của chúng với S cho ta biết điều gì?

3. Phá bế tắc: Loại bỏ những phần tử vô dụng

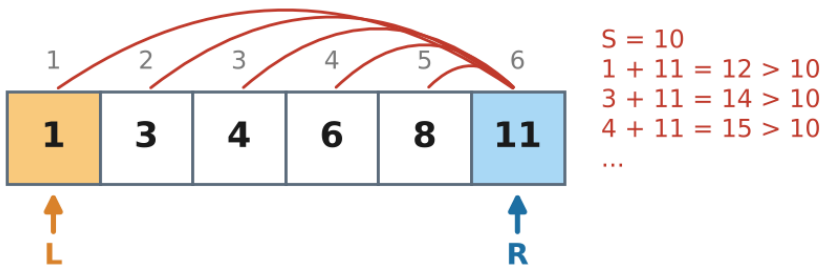
Bước 1 Nhìn vào hai đầu dãy

Xét ví dụ đầu tiên với S = 10. Phần tử nhỏ nhất là $a_1 = 1$, lớn nhất là $a_6 = 11$. Tổng $a_1 + a_6 = 12 > 10$: Quá lớn.

Bây giờ hãy hỏi về số 11: **Có số nào trong dãy ghép với 11 để được tổng 10 không?** Bạn nhỏ nhất của 11 là số 1, mà ngay cả 1 + 11 cũng đã vượt 10. Mọi bạn khác của 11 đều lớn hơn

Lê Thành Nam

hoặc bằng 1, nên tổng còn lớn hơn nữa. Vậy số 11 **không thể** nằm trong cặp đáp án nào. Ta gọi nó là phần tử **vô dụng** và gạch bỏ nó.



Hình 1.4. *Mọi cặp chứa số 11 đều có tổng vượt 10, vì ngay cặp nhỏ nhất (1, 11) đã vượt. Số 11 bị loại*

Tương tự, nếu $a_1 + a_n < S$ thì số a_1 vô dụng: Bạn lớn nhất của nó là a_n, mà ngay cả với bạn lớn nhất, tổng vẫn chưa đủ S. Mọi bạn khác đều nhỏ hơn hoặc bằng a_n nên tổng càng thiếu.

NHẬN XÉT THEN CHỐT

Gọi $t = a_1 + a_n$ (tổng của số nhỏ nhất và số lớn nhất). Có đúng ba khả năng:

- **t = S:** Tìm được đáp án (1, n).
- **t > S:** Số lớn nhất a_n vô dụng, vì a_n cộng với số nhỏ nhất đã vượt S. Loại a_n.
- **t < S:** Số nhỏ nhất a_1 vô dụng, vì a_1 cộng với số lớn nhất vẫn chưa đạt S. Loại a_1.

Dù rơi vào trường hợp nào, sau **một** phép so sánh ta hoặc có đáp án, hoặc loại được **một phần tử cùng với mọi cặp chứa nó**.

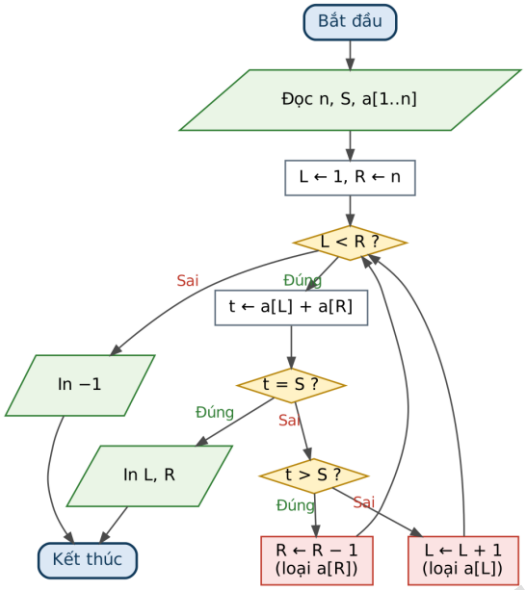
Bước 2 Lặp lại trên phần còn lại: Hai con trỏ

Sau khi loại một phần tử ở đầu dãy hoặc cuối dãy, phần còn lại vẫn là một **đoạn liên tiếp** đã sắp xếp, và ta lại có đúng bài toán cũ trên đoạn ngắn hơn. Vậy chỉ cần nhớ đoạn còn lại bằng hai chỉ số: L là đầu trái, R là đầu phải. Ban đầu L = 1, R = n. Mỗi lượt:

1. Tính $t = a_L + a_R$.
2. Nếu t = S: In (L, R) và dừng.
3. Nếu t > S: Loại a_R, tức là R giảm 1.
4. Nếu t < S: Loại a_L, tức là L tăng 1.

Thuật toán dừng khi tìm được đáp án hoặc khi L = R (đoạn còn một phần tử, không còn cặp nào). Hai chỉ số L và R được gọi là **hai con trỏ**: Chúng xuất phát từ hai đầu dãy và tiến về phía nhau, **không bao giờ quay đầu**.

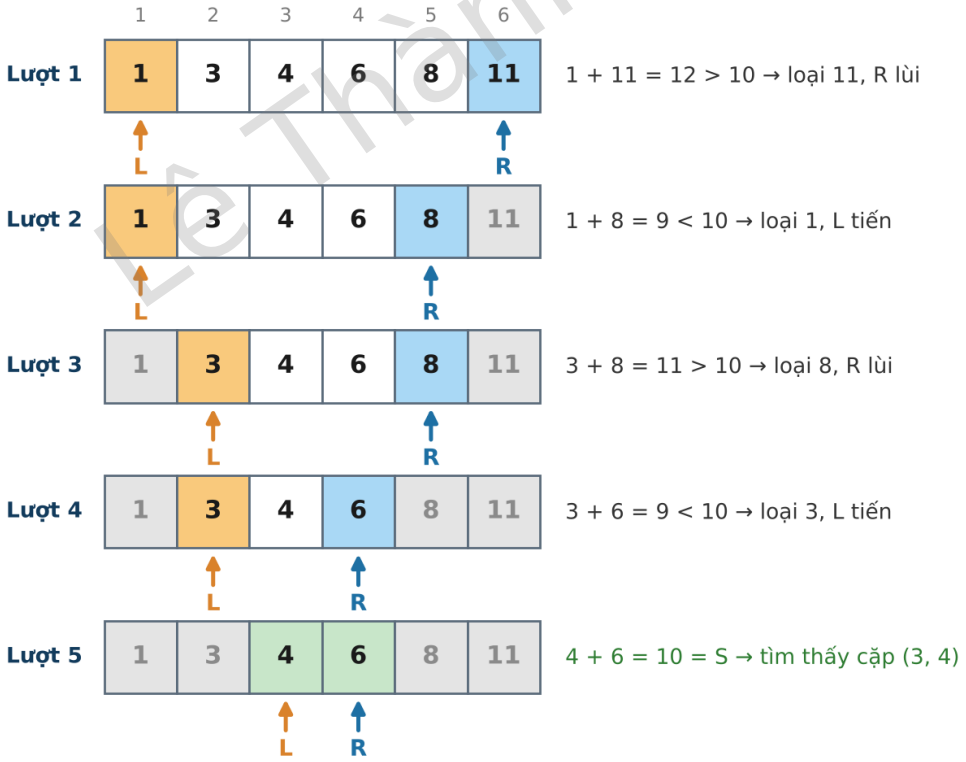
Lê Thanh Nam



Hình 1.5. Sơ đồ khối của thuật toán hai con trỏ. Hai ô đỏ là hai cách loại bỏ một phần tử

Bước 3 Chạy tay trên ví dụ

Với ví dụ 1 (S = 10), thuật toán chạy năm lượt. Hãy tự chạy trên giấy trước khi xem hình: Ở mỗi lượt, em chỉ cần so sánh một tổng với S và quyết định con trỏ nào di chuyển.



Hình 1.6. Chạy tay hai con trỏ với S = 10. Ô xám là phần tử đã bị loại, không bao giờ được xét lại

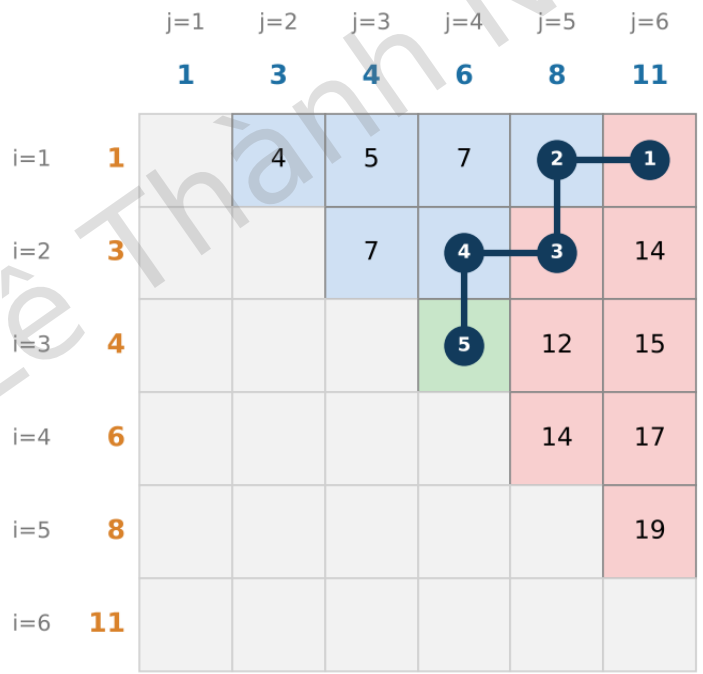
Lê Thành Nam

Lượt	L	R	a[L] + a[R]	Quyết định
1	1	6	1 + 11 = 12	Lớn hơn 10: Số 11 vô dụng, R giảm còn 5.
2	1	5	1 + 8 = 9	Nhỏ hơn 10: Số 1 vô dụng, L tăng lên 2.
3	2	5	3 + 8 = 11	Lớn hơn 10: Số 8 vô dụng, R giảm còn 4.
4	2	4	3 + 6 = 9	Nhỏ hơn 10: Số 3 vô dụng, L tăng lên 3.
5	3	4	4 + 6 = 10	**Bằng 10: In 3 4.**

Chỉ năm phép so sánh, trong khi cách ngây thơ phải thử 10 cặp (từ (1, 2) tới (3, 4)) mới gặp đáp án. Với ví dụ 3 (S = 20), tổng $a_L + a_R$ luôn nhỏ hơn 20 nên L tăng liên tục từ 1 tới 6 và gặp R; thuật toán in −1 sau đúng 5 lượt.

Bước 4 Vì sao không bỏ sót đáp án? Nhìn bằng bảng cặp

Để thấy rõ thuật toán đã "bỏ qua" những cặp nào, hãy kẻ một bảng: Hàng i, cột j, ô (i, j) ghi tổng $a_i + a_j$ (chỉ xét i < j, tức là nửa trên đường chéo). Vì dãy tăng, trong mỗi hàng các tổng tăng dần từ trái sang phải, trong mỗi cột các tổng tăng dần từ trên xuống dưới.



Hình 1.7. Bảng tổng các cặp với S = 10: Đỏ là tổng lớn hơn S, xanh dương là nhỏ hơn S, xanh lá là bằng S. Đường đi đánh số là năm lượt của hai con trỏ

Con trỏ bắt đầu ở ô (1, 6), góc trên bên phải. Ở lượt 1, tổng 12 > 10. Ô (1, 6) là ô **nhỏ nhất** của cột 6 (vì mọi ô khác trong cột 6 nằm phía dưới, tổng lớn hơn). Ô nhỏ nhất đã đỏ thì **cả cột 6 đều đỏ**: Ta xóa cả cột và đi sang trái. Ở lượt 2, tổng 9 < 10. Ô (1, 5) là ô **lớn nhất** của hàng

Lê Thành Nam

1 còn lại (các ô bên trái nhỏ hơn). Ô lớn nhất đã xanh dương thì **cả hàng 1 đều xanh dương**: Ta xóa cả hàng và đi xuống.

Cứ thế, đường đi là một **bậc thang** đi từ góc trên bên phải xuống dưới hoặc sang trái. Mỗi bước xóa trọn một hàng hoặc một cột mà không cần nhìn vào từng ô trong đó.

VÌ SAO ĐÚNG? CHỨNG MINH BẰNG BẤT BIẾN

Ta chứng minh khẳng định sau luôn đúng ở đầu mỗi lượt (gọi là **bất biến**): **Nếu tồn tại cặp đáp án (i, j), thì tồn tại một cặp đáp án nằm trong đoạn hiện tại, tức là $L \le i < j \le R$.**

- Ban đầu L = 1, R = n: Hiển nhiên đúng.
- Giả sử bất biến đúng ở đầu một lượt và có cặp đáp án (i, j) với $L \le i < j \le R$. Nếu $a_L + a_R > S$, ta khẳng định $j \ne R$: Vì nếu $j = R$ thì $a_i + a_R \ge a_L + a_R > S$ (do $a_i \ge a_L$), mâu thuẫn. Vậy $j \le R - 1$ và cặp (i, j) vẫn nằm trong đoạn mới $[L, R - 1]$.
- Tương tự, nếu $a_L + a_R < S$ thì $i \ne L$, nên cặp đáp án vẫn nằm trong đoạn mới $[L + 1, R]$.
- Khi thuật toán dừng vì L = R, đoạn không còn cặp nào. Theo bất biến, điều đó chỉ xảy ra khi **không tồn tại** đáp án nào cả. Vậy in −1 là đúng.

Lập luận "bất biến" này là công cụ chứng minh quan trọng nhất của cả cuốn sách. Ở mỗi bài học sau, em sẽ gặp lại nó dưới nhiều hình thức: Ta chỉ ra một điều luôn đúng trước và sau mỗi lần con trỏ di chuyển, rồi suy ra kết quả cuối cùng đúng.

Bước 5 Đếm số phép tính: Vì sao là O(n)?

ĐẾM SỐ PHÉP TÍNH

- Mỗi lượt, hoặc thuật toán dừng, hoặc **đúng một** trong hai con trỏ di chuyển một bước về phía con trỏ kia. Do đó hiệu R − L giảm đúng 1 sau mỗi lượt không dừng.
- Ban đầu R − L = n − 1, và thuật toán dừng muộn nhất khi R − L = 0. Vậy có **không quá n − 1 lượt**.
- Mỗi lượt gồm một phép cộng, tối đa hai phép so sánh và một phép tăng hoặc giảm: O(1). Tổng cộng **O(n)**.
- Với $n = 10^6$: Không quá 10^6 lượt, chạy trong vài mili giây. Phần lớn thời gian của chương trình là để **đọc dữ liệu**, chứ không phải để tìm cặp.

Hãy so sánh: Cách ngây thơ cần $5 \cdot 10^{11}$ phép thử; tìm kiếm nhị phân cần khoảng $2 \cdot 10^7$; hai con trỏ cần không quá 10^6. Khoảng cách giữa con số đầu và con số cuối là **năm trăm nghìn lần**.

Lê Thanh Nam

Bước 6 Chương trình hoàn chỉnh

TONGHAI.CPP: Hai con trỏ O(n)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("TONGHAI.INP", "r")) {
        freopen("TONGHAI.INP", "r", stdin);
        freopen("TONGHAI.OUT", "w", stdout);
    }
    int n;
    long long s;
    cin >> n >> s;
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    int l = 1, r = n;
    while (l < r) {
        long long t = a[l] + a[r];
        if (t == s) {
            cout << l << ' ' << r << '\n';
            return 0;
        }
        if (t > s) r--;
        else l++;
    }
    cout << -1 << '\n';
    return 0;
}
```

Lê Thành Nam

Có ba chi tiết đáng để ý trong chương trình. Thứ nhất, điều kiện vòng lặp là `l < r` chứ không phải `l <= r`: Đề yêu cầu hai chỉ số khác nhau, nên khi L = R thì không còn cặp nào. Thứ hai, biến t có kiểu long long. Thứ ba, ta đọc dữ liệu bằng cin sau khi tắt đồng bộ với scanf (hai dòng `ios_base::sync_with_stdio(false)` và `cin.tie(nullptr)`), vì với 10^6 số, cách đọc chậm có thể tốn nhiều thời gian hơn cả thuật toán.

Cách làm	Thời gian	Bộ nhớ phụ	Ghi chú
Thử mọi cặp	$O(n^2)$	O(1)	Chỉ chạy được $n \le 10^4$
Tìm kiếm nhị phân	O(n log n)	O(1)	Cần tìm đúng một giá trị
Hai con trỏ	O(n)	O(1)	**Đơn giản nhất, nhanh nhất**

4. Đếm số cặp có tổng không vượt quá S: Bài toán DEMCAP

Bài toán tìm một cặp chỉ cần thuật toán dừng lại khi gặp đáp án. Bây giờ ta đổi câu hỏi thành **đếm**, và em sẽ thấy hai con trỏ còn mạnh hơn nữa: Mỗi lượt không chỉ loại bỏ mà còn **đếm cả loạt** cặp cùng lúc.

BÀI TOÁN. ĐẾM CẶP [DEMCAP]

Cho dãy n số nguyên đã sắp xếp không giảm $a_1 \le ... \le a_n$ và số nguyên S. Đếm số cặp chỉ số (i, j) với $i < j$ và $a_i + a_j \le S$.

Dữ liệu vào: Dòng đầu gồm n, S ($1 \le n \le 10^6$, $|S| \le 2 \cdot 10^9$). Dòng thứ hai gồm n số đã sắp xếp, $|a_i| \le 10^9$.

Kết quả: Số cặp thỏa mãn.

DEMCAP.INP	DEMCAP.OUT	Giải thích
6 9 1 2 4 5 7 9	8	Các cặp: 1+2, 1+4, 1+5, 1+7, 2+4, 2+5, 2+7, 4+5.
3 0 1 2 3	0	Không có cặp nào.

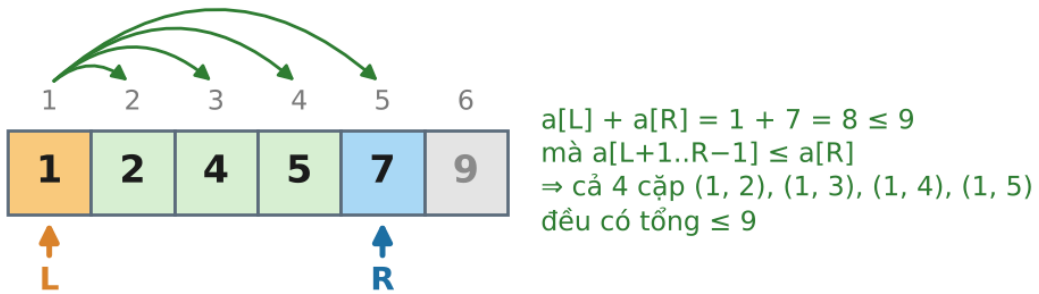
Đáp án có thể lớn tới $n(n-1)/2 \approx 5 \cdot 10^{11}$, vượt kiểu int. Biến đếm phải là long long.

Cách ngây thơ và bế tắc

Cách ngây thơ vẫn là hai vòng lặp, đếm mọi cặp có tổng ≤ S, độ phức tạp $O(n^2)$. Lần này nó không thể dừng sớm, vì phải đếm hết. Với $n = 10^6$, cách này cần đủ $5 \cdot 10^{11}$ phép thử trong **mọi** trường hợp. Ngay cả cách tìm kiếm nhị phân (với mỗi i, tìm vị trí j lớn nhất có $a_j \le S - a_i$) cũng cần O(n log n). Ta muốn O(n).

Nhận xét: Một phép so sánh, cả loạt cặp

Xét đoạn [L, R]. Nếu $a_L + a_R \le S$ thì với mọi j thuộc [L + 1, R], ta có $a_j \le a_R$, nên $a_L + a_j \le a_L + a_R \le S$. Nghĩa là **cả R − L cặp** (L, L + 1), (L, L + 2), ..., (L, R) đều thỏa mãn. Đếm chúng một lần, cộng R − L vào đáp án, rồi loại a_L (vì mọi cặp của a_L trong đoạn đã được đếm xong).



Hình 1.8. *Khi a[L] + a[R] ≤ S, mọi cặp (L, j) với L < j ≤ R đều thỏa mãn: Đếm cả R − L cặp trong một bước*

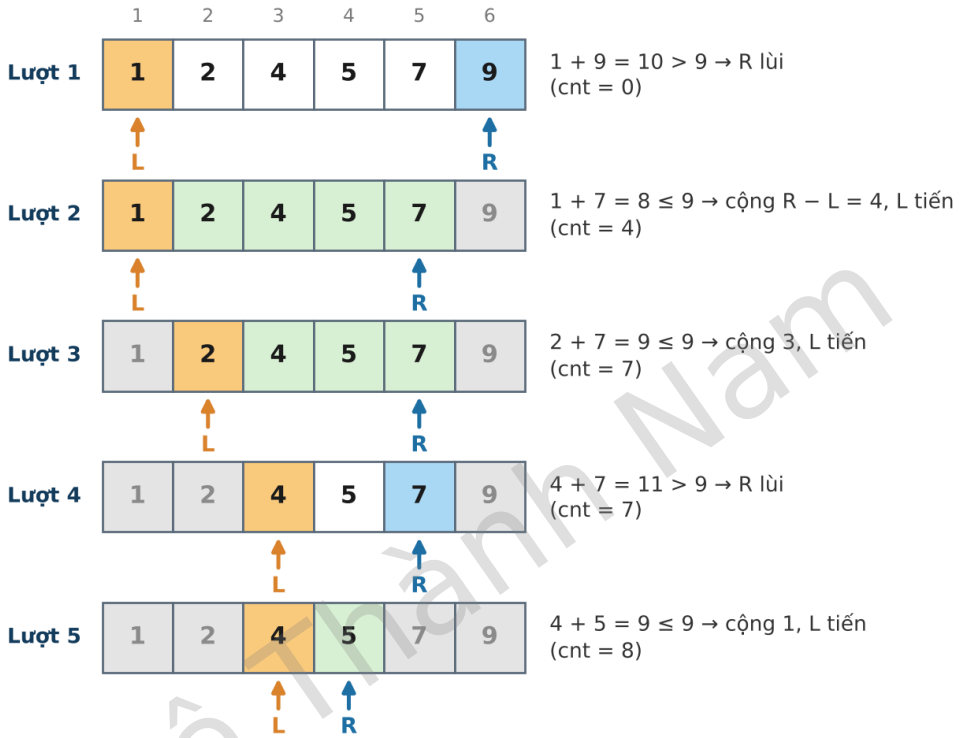
Nếu $a_L + a_R > S$ thì như bài trước, a_R cộng với số nhỏ nhất còn lại đã vượt S, nên a_R không tạo cặp thỏa mãn nào với các phần tử trong đoạn: Loại a_R.

Lê Thanh Nam

THUẬT TOÁN ĐẾM CẶP CÓ TỔNG ≤ S

Bắt đầu với L = 1, R = n, cnt = 0. Trong khi L < R:

- Nếu $a_L + a_R \le S$: cnt tăng thêm **R − L**, rồi L tăng 1.
- Ngược lại: R giảm 1.



Hình 1.9. *Chạy tay thuật toán đếm với S = 9. Ô xanh lá là các cặp được đếm cùng lúc ở lượt đó*

Lượt	L	R	a[L] + a[R]	Cộng thêm	Các cặp được đếm
1	1	6	1 + 9 = 10	0	(không có), R lùi
2	1	5	1 + 7 = 8	4	(1,2), (1,3), (1,4), (1,5)
3	2	5	2 + 7 = 9	3	(2,3), (2,4), (2,5)
4	3	5	4 + 7 = 11	0	(không có), R lùi
5	3	4	4 + 5 = 9	1	(3,4)
Kết thúc	4	4	| **8** |		

Lê Thành Nam

VÌ SAO ĐẾM ĐỦ VÀ KHÔNG TRÙNG?

Mỗi cặp (i, j) thỏa mãn được đếm **đúng một lần**, vào lượt mà con trỏ trái L dừng ở i và $a_L + a_R \le S$ lần đầu tiên. Ta cần hai điều:

- **Không thiếu:** Khi L dừng ở i và được đếm, R lúc đó là chỉ số lớn nhất còn lại; mọi $j > R$ đã bị loại vì có một $L' \le i$ với $a_{L'} + a_j > S$, nên $a_i + a_j \ge a_{L'} + a_j > S$, tức là các cặp (i, j) với $j > R$ đều không thỏa mãn. Còn mọi j trong $[i + 1, R]$ đều thỏa mãn và được đếm.
- **Không trùng:** Sau khi đếm, L tăng qua i và không bao giờ quay lại, nên các cặp có phần tử nhỏ hơn là i không được đếm lần thứ hai.

DEMCAP.CPP: Đếm cặp có tổng ≤ S trong O(n)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("DEMCAP.INP", "r")) {
        freopen("DEMCAP.INP", "r", stdin);
        freopen("DEMCAP.OUT", "w", stdout);
    }
    int n;
    long long s;
    cin >> n >> s;
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    long long cnt = 0;
    int l = 1, r = n;
    while (l < r) {
        if (a[l] + a[r] <= s) {
            cnt += r - l;
            l++;
        } else {
            r--;
        }
    }
    cout << cnt << '\n';
    return 0;
}
```

Lê Thành Nam

Số lượt vẫn không quá $n - 1$ (mỗi lượt một con trỏ di chuyển), nên độ phức tạp vẫn là **O(n)**, dù đáp án có thể lên tới $5 \cdot 10^{11}$. Đây là điều kì diệu đầu tiên của hai con trỏ: **Số phép tính nhỏ hơn rất nhiều so với số cặp được đếm**, vì mỗi phép cộng $R - L$ đếm cả một loạt cặp.